

Projekt 1: EasyAI — Tic-tac-doh

Authors

- Patrick Bajorski
- Jan Banasik

Wprowadzenie

Raport przedstawia implementację probabilistycznego wariantu kółko-krzyżyk (Tic-tac-doh) oraz porównanie algorytmów przeszukiwania drzewa gry pod względem jakości decyzji i kosztu obliczeniowego.

Opis gry

Tic-tac-doh to wariant gry 3×3, w którym z prawdopodobieństwem 20% wykonany ruch „nie dochodzi do skutku” (na planszy nie pojawia się znacznik), a ruch przechodzi na przeciwnika. Wprowadzenie losowości powoduje, że wynik rozgrywki nie jest deterministyczny, a algorytmy nieuwzględniające ryzyka mogą podejmować decyzje nieoptymalne w sensie wartości oczekiwanej.

Implementacja gry

Grę zaimplementowano w oparciu o EasyAI jako rozgrywkę dwuosobową z planszą 3×3. Kluczowym założeniem było rozdzielenie losowości od symulacji w drzewie gry: losowy „fail” jest stosowany wyłącznie przy faktycznym wykonaniu wybranego ruchu, natomiast w trakcie obliczania ruchu (symulacji stanów) mechanizm pozostaje deterministyczny.

Funkcja oceny przyjmuje wartość –100 w przypadku przegranej i 0 w pozostałych stanach końcowych, co oznacza, że algorytmy priorytetyzują unikanie przegranej.

Porównywane algorytmy

W ramach projektu zaimplementowano i porównano cztery algorytmy przeszukiwania drzewa gry:

Negamax z odcięciem alfa-beta

Standardowy algorytm z biblioteki EasyAI. Negamax to wariant algorytmu Minimax, gdzie wartość pozycji jest negowana przy zmianie gracza, eliminując potrzebę osobnych faz minimalizacji i maksymalizacji. Odcięcie alfa-beta przyspiesza przeszukiwanie poprzez eliminację gałęzi drzewa, które nie mogą wpłynąć na wynik.

Negamax bez odcięcia alfa-beta

Własna implementacja zgodna z interfejsem EasyAI. Przeszukuje **pełne drzewo gry** bez jakichkolwiek odcięć. Daje identyczne wyniki jak Negamax z alfa-beta, ale jest znacznie wolniejsza, ponieważ eksploruje każdy węzeł.

SSS*

Algorytm SSS* z biblioteki EasyAI. Jest to algorytm best-first search, który przeszukuje drzewo gry w kolejności „najbardziej obiecujących” gałęzi. Teoretycznie odwiedza mniejszą liczbę węzłów niż alfa-beta, ale wymaga dodatkowej pamięci.

Expecti-Minimax z odcięciem alfa-beta

Własna implementacja zaprojektowana dla gier z elementem losowości. W odróżnieniu od standardowego Negamax, Expecti-Minimax **modeluje węzły losowe (chance nodes)** w drzewie gry.

Dla każdego ruchu algorytm oblicza wartość oczekiwaną:

$$V(\text{ruch}) = (1 - p) \cdot V(\text{sukces}) + p \cdot V(\text{porażka})$$

gdzie $p = 0,2$ to prawdopodobieństwo nieudanego ruchu. W przypadku porażki plansza pozostaje bez zmian, a kolej przechodzi do przeciwnika.

Odcięcie alfa-beta na węzłach losowych wykorzystuje technikę **Star1 pruning**, która ogranicza przeszukiwanie na podstawie pesymistycznych i optymistycznych oszacowań wartości oczekiwanej. Dzięki temu algorytm nie musi zawsze eksplorować obu gałęzi (sukces/porażka) dla każdego ruchu — jeśli znana jest wartość jednej gałęzi, można zawęzić granice alfa-beta dla drugiej.

Eksperymenty

We wszystkich eksperymencie gracze AI rozgrywają partie **AI vs AI** z wymianą gracza rozpoczynającego co partię. Ziarno generatora losowego ustawiono na 42. Implementacja: Python $\geq$ 3.11, biblioteka EasyAI.

Eksperyment 1: Negamax — porównanie głębokości przeszukiwania

Algorytm Negamax (z odcięciem alfa-beta) uruchamiany przy dwóch głębokościach (**4** i **8**) na wariacie deterministycznym i probabilistycznym. Rozegrano **100 partii** na konfigurację.

Głębokość	Wariant	Gracz 1	Gracz 2	Remisy
4	Deterministyczny	0	0	100
4	Probabilistyczny	42	24	34
8	Deterministyczny	0	0	100
8	Probabilistyczny	35	16	49

- W wariacie **deterministycznym** głębokość 4 wystarczy do optymalnej gry — wszystkie partie kończą się remisem.
- W wariacie **probabilistycznym** losowość powoduje zwycięstwa; przy głębszym przeszukiwaniu (głębokość 8) AI lepiej kompensuje nieudane ruchy — więcej remisów (49 vs 34).
- Procent nieudanych ruchów w obu konfiguracjach probabilistycznych wynosi $\sim$ 18%, zgodnie z oczekiwanymi 20%.

Eksperyment 2: Porównanie algorytmów z pomiarem czasu

Porównanie **Negamax z odcięciem alfa-beta**, **Negamax bez odcięcia alfa-beta** oraz **SSS*** przy głębokościach **2** i **6**, na obu wariantach gry. Rozegrano **50 partii** na konfigurację. Czas mierzony jako średni czas wyboru ruchu (zegar wysokiej rozdzielczości), uśredniony po wszystkich ruchach ze wszystkich partii.

Głębokość 2

Algorytm	Wariant	G1	G2	Rem	Śr. czas/ruch
Negamax (α - β)	Deterministyczny	50	0	0	93 μ s
Negamax (α - β)	Probabilistyczny	36	12	2	93 μ s
Negamax (bez α - β)	Deterministyczny	50	0	0	204 μ s
Negamax (bez α - β)	Probabilistyczny	36	12	2	207 μ s
SSS*	Deterministyczny	50	0	0	124 μ s
SSS*	Probabilistyczny	36	12	2	120 μ s

Głębokość 6

Algorytm	Wariant	G1	G2	Rem	Śr. czas/ruch
Negamax (α - β)	Deterministyczny	0	0	50	2,59 ms
Negamax (α - β)	Probabilistyczny	20	10	20	3,34 ms
Negamax (bez α - β)	Deterministyczny	0	0	50	61,02 ms
Negamax (bez α - β)	Probabilistyczny	17	14	19	72,53 ms
SSS*	Deterministyczny	0	0	50	3,93 ms
SSS*	Probabilistyczny	17	14	19	4,38 ms

- W wariacie deterministycznym oraz przy głębokości 2 wszystkie algorytmy dają identyczne wyniki jakościowe. Przy głębokości 6 w wariacie probabilistycznym wyniki różnią się nieznacznie (Negamax α - β : 20/10/20 vs Negamax bez α - β i SSS*: 17/14/19) — przyczyną jest różnica w tie-breakingu: easyAI Negamax korzysta z tablicy transpozycji, która

zmienia kolejność oceniania ruchów o identycznej wartości.

- Odcięcie alfa-beta redukuje czas z 61 ms do 2,59 ms przy głębokości 6 (**23,6x** przyspieszenie), zgodnie z teorią: alfa-beta w najlepszym przypadku redukuje liczbę węzłów z $O(b^d)$ do $O(b^{d/2})$.
- SSS* jest nieznacznie wolniejszy od Negamax (α - β) z powodu narzutu pamięciowego utrzymywanej listy otwartych węzłów.

Eksperyment 3: Expecti-Minimax z odcięciem alfa-beta

Porównanie **ExpectiMinimax (α - β)** z algorytmami z eksperymentu 2, przy tych samych ustawieniach (głębokości **2** i **6**, **50** partii na konfigurację).

Głębokość 2

Algorytm	Wariant	G1	G2	Rem	Śr. czas/ruch
Negamax (α - β)	Deterministyczny	50	0	0	93 μ s
Negamax (α - β)	Probabilistyczny	36	12	2	93 μ s
Negamax (bez α - β)	Deterministyczny	50	0	0	204 μ s
Negamax (bez α - β)	Probabilistyczny	36	12	2	207 μ s
SSS*	Deterministyczny	50	0	0	124 μ s
SSS*	Probabilistyczny	36	12	2	120 μ s
ExpectiMinimax (α - β)	Deterministyczny	50	0	0	630 μ s
ExpectiMinimax (α - β)	Probabilistyczny	36	12	2	499 μ s

Głębokość 6

Algorytm	Wariant	G1	G2	Rem	Śr. czas/ruch
Negamax (α - β)	Deterministyczny	0	0	50	2,59 ms
Negamax (α - β)	Probabilistyczny	20	10	20	3,34 ms
Negamax (bez α - β)	Deterministyczny	0	0	50	61,02 ms
Negamax (bez α - β)	Probabilistyczny	17	14	19	72,53 ms
SSS*	Deterministyczny	0	0	50	3,93 ms
SSS*	Probabilistyczny	17	14	19	4,38 ms
ExpectiMinimax (α - β)	Deterministyczny	0	0	50	1,46 s
ExpectiMinimax (α - β)	Probabilistyczny	19	10	21	1,97 s

- ExpectiMinimax jest znacząco wolniejszy od pozostałych algorytmów — przy głębokości 6 ok. **590x** wolniejszy od Negamax (α - β) i ok. **27x** wolniejszy od Negamax (bez α - β).
- Przyczyną jest konieczność rozważenia dwóch scenariuszy dla każdego ruchu (sukces i porażka), co efektywnie zwiększa współczynnik rozgałęzienia. Pruning Star1 częściowo ogranicza ten narzut — odcinając gałęzie porażki, gdy znane granice wartości oczekiwanej wykluczają wpływ na decyzję.
- Pod względem jakości decyzji w wariancie probabilistycznym ExpectiMinimax uzyskuje 21 remisów przy głębokości 6, wobec 20 dla Negamax (α - β) i 19 dla Negamax (bez α - β) oraz SSS* — różnice są niewielkie.

Napotkane problemy

1. **Separacja losowości od symulacji AI** — kluczowe było zapewnienie, że losowość (20% nieudanych ruchów) nie zaburza przeszukiwania drzewa gry. Rozwiązaniem było stosowanie losowego „fail” wyłącznie podczas faktycznego wykonania wybranego ruchu, a nie podczas symulacji stanów w trakcie obliczeń.
2. **Wydajność Expecti-Minimax** — algorytm jest ~590x wolniejszy od Negamax przy głębokości 6 (1,97 s vs 3,34 ms na ruch). W praktyce przekłada się to na duży czas łączny obliczeń: dla konfiguracji probabilistycznej (50 gier) suma czasów wyboru ruchu wyniosła ok. 856 s ($\approx$ 14 min). Podwójne rozgałęzienie na każdym węźle (sukces/porażka) dramatycznie zwiększa liczbę odwiedzanych węzłów. Mimo zastosowania pruningu Star1, narzut ten jest trudny do uniknięcia, co ogranicza maksymalną użyteczną głębokość przeszukiwania.
3. **Zgodność interfejsów** — własne implementacje algorytmów musiały być zgodne z interfejsem biblioteki wykorzystywanym do rozgrywek AI vs AI.

Podsumowanie

Projekt obejmował implementację probabilistycznej gry Tic-tac-doh oraz porównanie czterech algorytmów przeszukiwania drzewa gry: Negamax z odcięciem alfa-beta, Negamax bez odcięcia, SSS* oraz Expecti-Minimax z odcięciem alfa-beta.

Kluczowe wnioski:

- **Odcięcie alfa-beta** jest krytyczną optymalizacją — przyspiesza przeszukiwanie od $2\times$ (głębokość 2) do ponad $23\times$ (głębokość 6), bez wpływu na jakość decyzji.
- **SSS*** nie oferuje istotnej przewagi nad Negamax z alfa-beta w kontekście Tic-tac-doh — jest nieco wolniejszy i daje identyczne wyniki.
- **Głębsze przeszukiwanie** poprawia wyniki w wariancie probabilistycznym — AI lepiej kompensuje losowe porażki ruchów.
- **Expecti-Minimax** jako jedyny algorytm explicite modeluje losowość w drzewie gry. Osiąga nieznacznie lepsze wyniki w wariancie probabilistycznym (więcej remisów), ale kosztem dramatycznie większego czasu obliczeń ($\sim 590\times$ wolniejszy od Negamax z alfa-beta przy głębokości 6). W najbardziej kosztownej konfiguracji (głębokość 6, wariant probabilistyczny) łączny czas obliczeń dla 50 gier wyniósł ok. 14 minut, co istotnie ogranicza praktyczne zastosowanie algorytmu dla większych głębokości.
- W kontekście Tic-tac-doh, ze względu na stosunkowo płytke drzewo gry (maks. 9 ruchów), różnice w jakości decyzji między Negamax ($\alpha\text{-}\beta$) a Expecti-Minimax nie są znaczące. Algorytm Expecti-Minimax miałby większy wpływ w grach z głębszym drzewem i wyższym prawdopodobieństwem losowych zdarzeń.